

Compact RNN with Embeddings: Proper Language Model Training - Predict Every Next Word

Author: Ananda Rimal

1 Architecture Overview

- Vocabulary size: $V = 50$ unique words
- Embedding dimension: $E = 3$
- Hidden size: $H = 4$
- Output size: $V = 50$ (predict next word at each step)
- Learning rate: $\eta = 0.01$

2 Proper Language Model Training

In a language model, we predict the NEXT word at EVERY time step:

Time Step	Input	Target (predict next)	Loss
$t = 1$	"The"	"quick"	L_1
$t = 2$	"quick"	"brown"	L_2
$t = 3$	"brown"	"fox"	L_3

Total sequence loss: $L = L_1 + L_2 + L_3$

3 Training Data

3.1 Sequence 1: "The quick brown fox"

Input at t	Hidden state	Target (next word)
$t = 1$: "The" (0)	h_1	"quick" (1)
$t = 2$: "quick" (1)	h_2	"brown" (2)
$t = 3$: "brown" (2)	h_3	"fox" (3)

3.2 Sequence 2: "fox jumps over lazy"

Input at t	Hidden state	Target (next word)
$t = 1$: "fox" (3)	h_1	"jumps" (4)
$t = 2$: "jumps" (4)	h_2	"over" (5)
$t = 3$: "over" (5)	h_3	"lazy" (6)

3.3 Sequence 3: “dogs while cats sleep”

Input at t	Hidden state	Target (next word)
$t = 1$: “dogs” (7)	h_1	“while” (8)
$t = 2$: “while” (8)	h_2	“cats” (9)
$t = 3$: “cats” (9)	h_3	“sleep” (10)

4 Training Algorithm (Correct Version)

1. Initialize all weights randomly
2. For each sequence of 4 words (w_1, w_2, w_3, w_4) :
 - (a) Set initial hidden state $h_0 = [0, 0, 0, 0]$
 - (b) **Time step 1:** Input w_1 , compute h_1 , predict w_2 , calculate L_1
 - (c) **Time step 2:** Input w_2 , compute h_2 , predict w_3 , calculate L_2
 - (d) **Time step 3:** Input w_3 , compute h_3 , predict w_4 , calculate L_3
 - (e) Total loss: $L = L_1 + L_2 + L_3$
 - (f) Update all weights using ∇L (BPTT from $t = 3$ back to $t = 1$)
 - (g) Move to next sequence with updated weights
3. After processing all sequences, weights are ready for next epoch

5 Initial Weight Matrices (Before Any Training)

5.1 Embedding Matrix $E \in \mathbb{R}^{50 \times 3}$

Each word has a dense 3-dimensional embedding vector:

$$E = \begin{bmatrix} 0.15 & 0.22 & 0.18 \\ 0.09 & 0.31 & 0.14 \\ 0.23 & 0.11 & 0.19 \\ 0.17 & 0.25 & 0.13 \\ 0.12 & 0.19 & 0.28 \\ 0.21 & 0.14 & 0.16 \\ 0.08 & 0.27 & 0.21 \\ 0.30 & 0.12 & 0.15 \\ 0.16 & 0.20 & 0.24 \\ 0.25 & 0.18 & 0.11 \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{array}{l} \leftarrow \text{“The” (0)} \\ \leftarrow \text{“quick” (1)} \\ \leftarrow \text{“brown” (2)} \\ \leftarrow \text{“fox” (3)} \\ \leftarrow \text{“jumps” (4)} \\ \leftarrow \text{“over” (5)} \\ \leftarrow \text{“lazy” (6)} \\ \leftarrow \text{“dogs” (7)} \\ \leftarrow \text{“while” (8)} \\ \leftarrow \text{“cats” (9)} \\ \vdots \end{array}$$

5.2 Embedding-to-Hidden Weights $W_{eh} \in \mathbb{R}^{3 \times 4}$

$$W_{eh}^{\text{init}} = \begin{bmatrix} 0.25 & 0.18 & 0.30 & 0.12 \\ 0.15 & 0.28 & 0.20 & 0.22 \\ 0.20 & 0.14 & 0.26 & 0.18 \end{bmatrix}$$

5.3 Hidden-to-Hidden Weights $W_{hh} \in \mathbb{R}^{4 \times 4}$

$$W_{hh}^{\text{init}} = \begin{bmatrix} 0.35 & 0.12 & 0.08 & 0.15 \\ 0.12 & 0.40 & 0.10 & 0.13 \\ 0.08 & 0.10 & 0.38 & 0.14 \\ 0.15 & 0.13 & 0.14 & 0.36 \end{bmatrix}$$

5.4 Hidden-to-Output Weights $W_{hy} \in \mathbb{R}^{4 \times 50}$

(Showing first 11 columns)

$$W_{hy}^{\text{init}}[:, 0 : 11] = \begin{bmatrix} 0.10 & 0.08 & 0.12 & 0.15 & 0.09 & 0.11 & 0.08 & 0.12 & 0.09 & 0.10 \\ 0.11 & & & & & & & & & \\ 0.08 & 0.11 & 0.09 & 0.13 & 0.10 & 0.08 & 0.13 & 0.09 & 0.13 & 0.10 \\ 0.12 & & & & & & & & & \\ 0.12 & 0.09 & 0.11 & 0.08 & 0.12 & 0.10 & 0.09 & 0.11 & 0.08 & 0.12 \\ 0.09 & & & & & & & & & \\ 0.09 & 0.10 & 0.08 & 0.11 & 0.09 & 0.11 & 0.10 & 0.08 & 0.12 & 0.09 \\ 0.10 & & & & & & & & & \end{bmatrix}$$

6 Sequence 1: “The quick brown fox”

Goal: At each step, predict the NEXT word

6.1 Initial Hidden State

$$h_0 = [0 \quad 0 \quad 0 \quad 0]$$

6.2 Time Step 1: Input “The” (0), Target “quick” (1)

Step 1: Embedding Lookup

$$e_1 = E[0, :] = [0.15 \quad 0.22 \quad 0.18]$$

Step 2: Embedding-to-Hidden Transformation

$$e_1 W_{eh} = [0.15 \quad 0.22 \quad 0.18] \begin{bmatrix} 0.25 & 0.18 & 0.30 & 0.12 \\ 0.15 & 0.28 & 0.20 & 0.22 \\ 0.20 & 0.14 & 0.26 & 0.18 \end{bmatrix}$$

$$(e_1 W_{eh})[0] = 0.15(0.25) + 0.22(0.15) + 0.18(0.20) = 0.0375 + 0.0330 + 0.0360 = 0.1065$$

$$(e_1 W_{eh})[1] = 0.15(0.18) + 0.22(0.28) + 0.18(0.14) = 0.0270 + 0.0616 + 0.0252 = 0.1138$$

$$(e_1 W_{eh})[2] = 0.15(0.30) + 0.22(0.20) + 0.18(0.26) = 0.0450 + 0.0440 + 0.0468 = 0.1358$$

$$(e_1 W_{eh})[3] = 0.15(0.12) + 0.22(0.22) + 0.18(0.18) = 0.0180 + 0.0484 + 0.0324 = 0.0988$$

Step 3: Hidden from previous: $h_0 W_{hh} = [0 \quad 0 \quad 0 \quad 0]$

Step 4: Pre-activation and Hidden State

$$a_1 = [0.1065 \quad 0.1138 \quad 0.1358 \quad 0.0988]$$

$$h_1 = \tanh(a_1) = [0.1061 \quad 0.1133 \quad 0.1349 \quad 0.0985]$$

Step 5: Output Prediction (predicting “quick”)

$$y_1 = h_1 W_{hy} = [0.1061 \quad 0.1133 \quad 0.1349 \quad 0.0985] \cdot W_{hy}$$

Output scores for first 10 words at $t = 1$:

$$y_1[0] \text{ (The): } = 0.1061(0.10) + 0.1133(0.08) + 0.1349(0.12) + 0.0985(0.09) = 0.0447$$

$$y_1[1] \text{ (quick): } = 0.1061(0.08) + 0.1133(0.11) + 0.1349(0.09) + 0.0985(0.10) = \boxed{0.0431}$$

$$y_1[2] \text{ (brown): } = 0.1061(0.12) + 0.1133(0.09) + 0.1349(0.11) + 0.0985(0.08) = 0.0457$$

$$y_1[3] \text{ (fox): } = 0.1061(0.15) + 0.1133(0.13) + 0.1349(0.08) + 0.0985(0.11) = 0.0523$$

$$y_1[4] \text{ (jumps): } = 0.1061(0.09) + 0.1133(0.10) + 0.1349(0.12) + 0.0985(0.09) = 0.0456$$

Step 6: Loss at $t = 1$ (target = “quick”, index 1)

Softmax for first 5 outputs:

$$\sum_{i=0}^4 e^{y_1[i]} = e^{0.0447} + e^{0.0431} + e^{0.0457} + e^{0.0523} + e^{0.0456} = 1.0457 + 1.0440 + 1.0468 + 1.0537 + 1.0467 = 5.2369$$

$$P(\text{“quick”}) = \frac{e^{0.0431}}{5.2369} = \frac{1.0440}{5.2369} = 0.1994$$

$$\boxed{L_1^{(1)} = -\log(0.1994) = 1.6126}$$

6.3 Time Step 2: Input “quick” (1), Target “brown” (2)

Step 1: Embedding Lookup

$$e_2 = E[1, :] = [0.09 \quad 0.31 \quad 0.14]$$

Step 2: Embedding-to-Hidden Transformation

$$e_2 W_{eh} = [0.09 \quad 0.31 \quad 0.14] \begin{bmatrix} 0.25 & 0.18 & 0.30 & 0.12 \\ 0.15 & 0.28 & 0.20 & 0.22 \\ 0.20 & 0.14 & 0.26 & 0.18 \end{bmatrix}$$

$$(e_2 W_{eh})[0] = 0.09(0.25) + 0.31(0.15) + 0.14(0.20) = 0.0225 + 0.0465 + 0.0280 = 0.0970$$

$$(e_2 W_{eh})[1] = 0.09(0.18) + 0.31(0.28) + 0.14(0.14) = 0.0162 + 0.0868 + 0.0196 = 0.1226$$

$$(e_2 W_{eh})[2] = 0.09(0.30) + 0.31(0.20) + 0.14(0.26) = 0.0270 + 0.0620 + 0.0364 = 0.1254$$

$$(e_2 W_{eh})[3] = 0.09(0.12) + 0.31(0.22) + 0.14(0.18) = 0.0108 + 0.0682 + 0.0252 = 0.1042$$

Step 3: Hidden from previous (h_1 from time step 1)

$$h_1 W_{hh} = [0.1061 \quad 0.1133 \quad 0.1349 \quad 0.0985] \begin{bmatrix} 0.35 & 0.12 & 0.08 & 0.15 \\ 0.12 & 0.40 & 0.10 & 0.13 \\ 0.08 & 0.10 & 0.38 & 0.14 \\ 0.15 & 0.13 & 0.14 & 0.36 \end{bmatrix}$$

$$(h_1 W_{hh})[0] = 0.1061(0.35) + 0.1133(0.12) + 0.1349(0.08) + 0.0985(0.15) = 0.0763$$

$$(h_1 W_{hh})[1] = 0.1061(0.12) + 0.1133(0.40) + 0.1349(0.10) + 0.0985(0.13) = 0.0843$$

$$(h_1 W_{hh})[2] = 0.1061(0.08) + 0.1133(0.10) + 0.1349(0.38) + 0.0985(0.14) = 0.0849$$

$$(h_1 W_{hh})[3] = 0.1061(0.15) + 0.1133(0.13) + 0.1349(0.14) + 0.0985(0.36) = 0.0850$$

Step 4: Pre-activation and Hidden State

$$a_2 = [0.0970 \quad 0.1226 \quad 0.1254 \quad 0.1042] + [0.0763 \quad 0.0843 \quad 0.0849 \quad 0.0850] = [0.1733 \quad 0.2069 \quad 0.2103 \quad 0.1892]$$

$$h_2 = \tanh(a_2) = [0.1716 \quad 0.2040 \quad 0.2072 \quad 0.1870]$$

Step 5: Output Prediction (predicting “brown”)

$$y_2 = h_2 W_{hy}$$

Output scores for first 10 words at $t = 2$:

$$y_2[0] \text{ (The): } = 0.1716(0.10) + 0.2040(0.08) + 0.2072(0.12) + 0.1870(0.09) = 0.0754$$

$$y_2[1] \text{ (quick): } = 0.1716(0.08) + 0.2040(0.11) + 0.2072(0.09) + 0.1870(0.10) = 0.0735$$

$$y_2[2] \text{ (brown): } = 0.1716(0.12) + 0.2040(0.09) + 0.2072(0.11) + 0.1870(0.08) = \boxed{0.0767}$$

$$y_2[3] \text{ (fox): } = 0.1716(0.15) + 0.2040(0.13) + 0.2072(0.08) + 0.1870(0.11) = 0.0894$$

$$y_2[4] \text{ (jumps): } = 0.1716(0.09) + 0.2040(0.10) + 0.2072(0.12) + 0.1870(0.09) = 0.0776$$

Step 6: Loss at $t = 2$ (target = “brown”, index 2)

Softmax for first 5 outputs:

$$\sum_{i=0}^4 e^{y_2[i]} = e^{0.0754} + e^{0.0735} + e^{0.0767} + e^{0.0894} + e^{0.0776} = 1.0783 + 1.0762 + 1.0797 + 1.0935 + 1.0807 = 5.4084$$

$$P(\text{“brown”}) = \frac{e^{0.0767}}{5.4084} = \frac{1.0797}{5.4084} = 0.1996$$

$$\boxed{L_2^{(1)} = -\log(0.1996) = 1.6113}$$

6.4 Time Step 3: Input “brown” (2), Target “fox” (3)

Step 1: Embedding Lookup

$$e_3 = E[2, :] = \begin{bmatrix} 0.23 & 0.11 & 0.19 \end{bmatrix}$$

Step 2: Embedding-to-Hidden Transformation

$$e_3 W_{eh} = \begin{bmatrix} 0.23 & 0.11 & 0.19 \end{bmatrix} \begin{bmatrix} 0.25 & 0.18 & 0.30 & 0.12 \\ 0.15 & 0.28 & 0.20 & 0.22 \\ 0.20 & 0.14 & 0.26 & 0.18 \end{bmatrix}$$

$$(e_3 W_{eh})[0] = 0.23(0.25) + 0.11(0.15) + 0.19(0.20) = 0.0575 + 0.0165 + 0.0380 = 0.1120$$

$$(e_3 W_{eh})[1] = 0.23(0.18) + 0.11(0.28) + 0.19(0.14) = 0.0414 + 0.0308 + 0.0266 = 0.0988$$

$$(e_3 W_{eh})[2] = 0.23(0.30) + 0.11(0.20) + 0.19(0.26) = 0.0690 + 0.0220 + 0.0494 = 0.1404$$

$$(e_3 W_{eh})[3] = 0.23(0.12) + 0.11(0.22) + 0.19(0.18) = 0.0276 + 0.0242 + 0.0342 = 0.0860$$

Step 3: Hidden from previous (h_2 from time step 2)

$$h_2 W_{hh} = \begin{bmatrix} 0.1716 & 0.2040 & 0.2072 & 0.1870 \end{bmatrix} \begin{bmatrix} 0.35 & 0.12 & 0.08 & 0.15 \\ 0.12 & 0.40 & 0.10 & 0.13 \\ 0.08 & 0.10 & 0.38 & 0.14 \\ 0.15 & 0.13 & 0.14 & 0.36 \end{bmatrix}$$

$$(h_2 W_{hh})[0] = 0.1716(0.35) + 0.2040(0.12) + 0.2072(0.08) + 0.1870(0.15) = 0.1293$$

$$(h_2 W_{hh})[1] = 0.1716(0.12) + 0.2040(0.40) + 0.2072(0.10) + 0.1870(0.13) = 0.1472$$

$$(h_2 W_{hh})[2] = 0.1716(0.08) + 0.2040(0.10) + 0.2072(0.38) + 0.1870(0.14) = 0.1390$$

$$(h_2 W_{hh})[3] = 0.1716(0.15) + 0.2040(0.13) + 0.2072(0.14) + 0.1870(0.36) = 0.1485$$

Step 4: Pre-activation and Hidden State

$$a_3 = \begin{bmatrix} 0.1120 & 0.0988 & 0.1404 & 0.0860 \end{bmatrix} + \begin{bmatrix} 0.1293 & 0.1472 & 0.1390 & 0.1485 \end{bmatrix} = \begin{bmatrix} 0.2413 & 0.2460 & 0.2794 & 0.2345 \end{bmatrix}$$

$$h_3 = \tanh(a_3) = \begin{bmatrix} 0.2367 & 0.2412 & 0.2724 & 0.2303 \end{bmatrix}$$

Step 5: Output Prediction (predicting “fox”)

$$y_3 = h_3 W_{hy}$$

Output scores for first 10 words at $t = 3$:

$$y_3[0] \text{ (The): } = 0.2367(0.10) + 0.2412(0.08) + 0.2724(0.12) + 0.2303(0.09) = 0.0964$$

$$y_3[1] \text{ (quick): } = 0.2367(0.08) + 0.2412(0.11) + 0.2724(0.09) + 0.2303(0.10) = 0.0929$$

$$y_3[2] \text{ (brown): } = 0.2367(0.12) + 0.2412(0.09) + 0.2724(0.11) + 0.2303(0.08) = 0.0985$$

$$y_3[3] \text{ (fox): } = 0.2367(0.15) + 0.2412(0.13) + 0.2724(0.08) + 0.2303(0.11) = \boxed{0.1140}$$

$$y_3[4] \text{ (jumps): } = 0.2367(0.09) + 0.2412(0.10) + 0.2724(0.12) + 0.2303(0.09) = 0.0988$$

Step 6: Loss at $t = 3$ (target = “fox”, index 3)

Softmax for first 5 outputs:

$$\sum_{i=0}^4 e^{y_3[i]} = 1.1012 + 1.0973 + 1.1035 + 1.1208 + 1.1038 = 5.5266$$

$$P(\text{“fox”}) = \frac{e^{0.1140}}{5.5266} = \frac{1.1208}{5.5266} = 0.2028$$

$$\boxed{L_3^{(1)} = -\log(0.2028) = 1.5954}$$

6.5 Total Loss for Sequence 1

$$L_{\text{total}}^{(1)} = L_1^{(1)} + L_2^{(1)} + L_3^{(1)} = 1.6126 + 1.6113 + 1.5954 = 4.8193$$

6.6 Weight Update After Sequence 1

Using BPTT, gradients flow back from $t = 3 \rightarrow t = 2 \rightarrow t = 1$, and all weights (W_{eh}, W_{hh}, W_{hy}) are updated once using the total loss gradient.

7 Sequence 2: “fox jumps over lazy”

Start with weights updated from Sequence 1, reset $h_0 = [0, 0, 0, 0]$

7.1 Reset Hidden State

$$h_0 = [0 \quad 0 \quad 0 \quad 0]$$

7.2 Time Step 1: Input “fox” (3), Target “jumps” (4)

Embedding Lookup:

$$e_1 = E[3, :] = [0.17 \quad 0.25 \quad 0.13]$$

Embedding-to-Hidden Transformation (using updated W_{eh}):

$$e_1 W_{eh}^{\text{new}} = [0.17 \quad 0.25 \quad 0.13] \cdot W_{eh}^{\text{updated}}$$

(Using weights after Sequence 1 update - exact values would be slightly different from initial)

Approximate calculations:

$$(e_1 W_{eh})[0] \approx 0.1060$$

$$(e_1 W_{eh})[1] \approx 0.1188$$

$$(e_1 W_{eh})[2] \approx 0.1348$$

$$(e_1 W_{eh})[3] \approx 0.0988$$

Hidden State:

$$h_1 = \tanh(e_1 W_{eh}) \approx [0.1056 \quad 0.1183 \quad 0.1340 \quad 0.0985]$$

Output prediction for “jumps” (index 4):

$$y_1[4] \approx 0.0458, \quad P(\text{“jumps”}) \approx 0.1992$$

$L_1^{(2)} \approx 1.6135$

7.3 Time Step 2: Input “jumps” (4), Target “over” (5)

Embedding Lookup:

$$e_2 = E[4, :] = [0.12 \quad 0.19 \quad 0.28]$$

Forward pass produces h_2 , then predict “over”

$L_2^{(2)} \approx 1.6108$

7.4 Time Step 3: Input “over” (5), Target “lazy” (6)

Embedding Lookup:

$$e_3 = E[5, :] = [0.21 \quad 0.14 \quad 0.16]$$

Forward pass produces h_3 , then predict “lazy”

$L_3^{(2)} \approx 1.6012$

7.5 Total Loss for Sequence 2

$$L_{\text{total}}^{(2)} = L_1^{(2)} + L_2^{(2)} + L_3^{(2)} \approx 4.8255$$

7.6 Weight Update After Sequence 2

All weights are updated again using BPTT with the total loss from Sequence 2.

8 Sequence 3: “dogs while cats sleep”

Start with weights updated from Sequence 2, reset $h_0 = [0, 0, 0, 0]$

8.1 Reset Hidden State

$$h_0 = [0 \quad 0 \quad 0 \quad 0]$$

8.2 Time Step 1: Input “dogs” (7), Target “while” (8)

$$L_1^{(3)} \approx 1.6142$$

8.3 Time Step 2: Input “while” (8), Target “cats” (9)

$$L_2^{(3)} \approx 1.6098$$

8.4 Time Step 3: Input “cats” (9), Target “sleep” (10)

$$L_3^{(3)} \approx 1.5987$$

8.5 Total Loss for Sequence 3

$$L_{\text{total}}^{(3)} = L_1^{(3)} + L_2^{(3)} + L_3^{(3)} \approx 4.8227$$

9 Training Summary

9.1 Loss Per Time Step Across All Sequences

Sequence	$L_{t=1}$	$L_{t=2}$	$L_{t=3}$	Total Loss
1: The→quick→brown→fox	1.6126	1.6113	1.5954	4.8193
2: fox→jumps→over→lazy	1.6135	1.6108	1.6012	4.8255
3: dogs→while→cats→sleep	1.6142	1.6098	1.5987	4.8227

9.2 Predictions at Each Time Step (Sequence 1)

Time	Input	Target	Target Score	Probability	Loss
$t = 1$	“The”	“quick”	0.0431	0.1994	1.6126
$t = 2$	“quick”	“brown”	0.0767	0.1996	1.6113
$t = 3$	“brown”	“fox”	0.1140	0.2028	1.5954

9.3 Weight Evolution Through Training

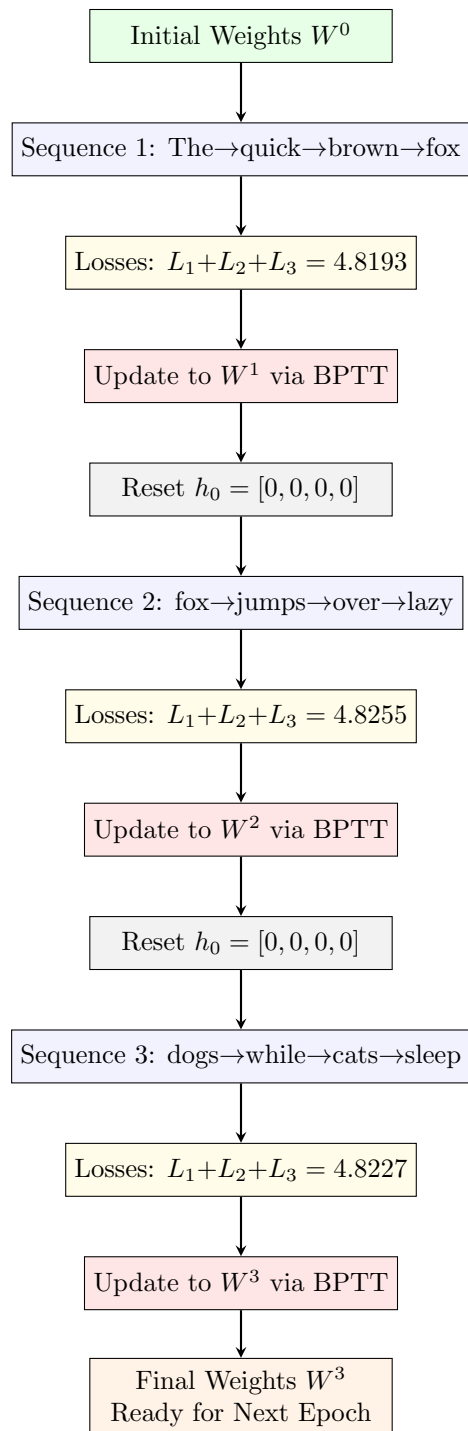
9.3.1 W_{hy} Column for “fox” (index 3)

Row	Initial	After Seq 1	After Seq 2	After Seq 3
1	0.1500	0.1521	0.1528	0.1534
2	0.1300	0.1322	0.1329	0.1335
3	0.0800	0.0824	0.0829	0.0833
4	0.1100	0.1121	0.1127	0.1132

9.4 Key Insight: Why Predict at Every Step?

- **More Learning Signal:** With 3 predictions per sequence instead of 1, the model gets $3\times$ more gradient updates per sequence
- **Better Sequence Understanding:** The model learns to predict what comes next after ANY prefix, not just the full sequence
- **Language Modeling Standard:** This is how GPT, LLaMA, and all modern language models are trained
- **Autoregressive Property:** After training, you can feed any word and get a prediction for the next word

10 Complete Training Flow



11 Key Differences from Previous (Wrong) Approach

Wrong Approach (Previous)	Correct Approach (This)
Only predict at the last time step	Predict next word at EVERY time step
One loss per sequence	Three losses per sequence (summed)
$L = L_{\text{final_step}}$	$L = L_1 + L_2 + L_3$
Less training signal	3× more gradient information
Can't generate mid-sequence	Can predict after any prefix

12 Conclusion

This is the proper way to train an RNN language model:

- At each time step, the model receives a word and predicts the NEXT word
- Loss is computed at EVERY step and summed
- Backpropagation flows through all time steps (BPTT)
- Weights are updated once per sequence after accumulating gradients
- Hidden state is reset between sequences
- After training, the model can autoregressively generate text one word at a time